



University of San Carlos | Department of
COMPUTER ENGINEERING

CpE 3104 – Microprocessors

Interfacing Parallel I/O (LCD)

UNIT 7: INTERFACING PARALLEL I/O DEVICES

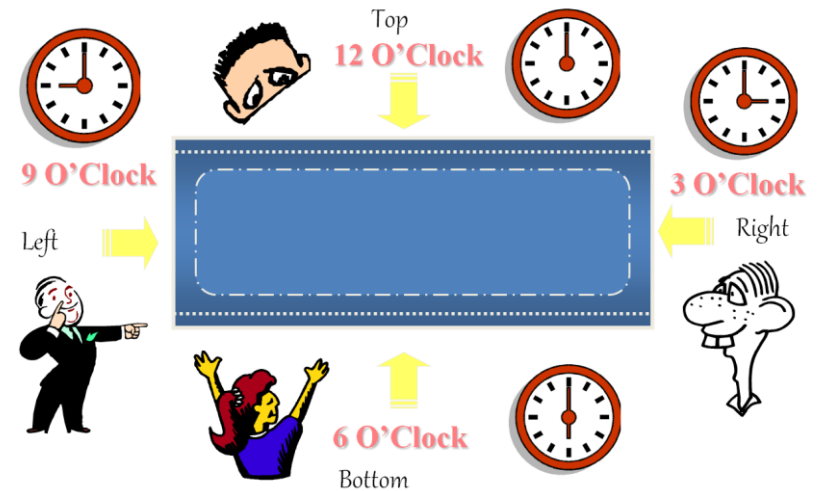
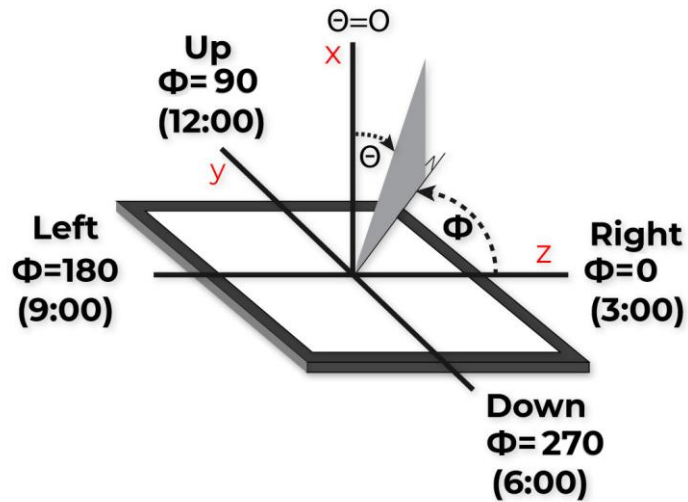
© 2025

Interfacing Displaytech 204a LCD to 8086 Microprocessor

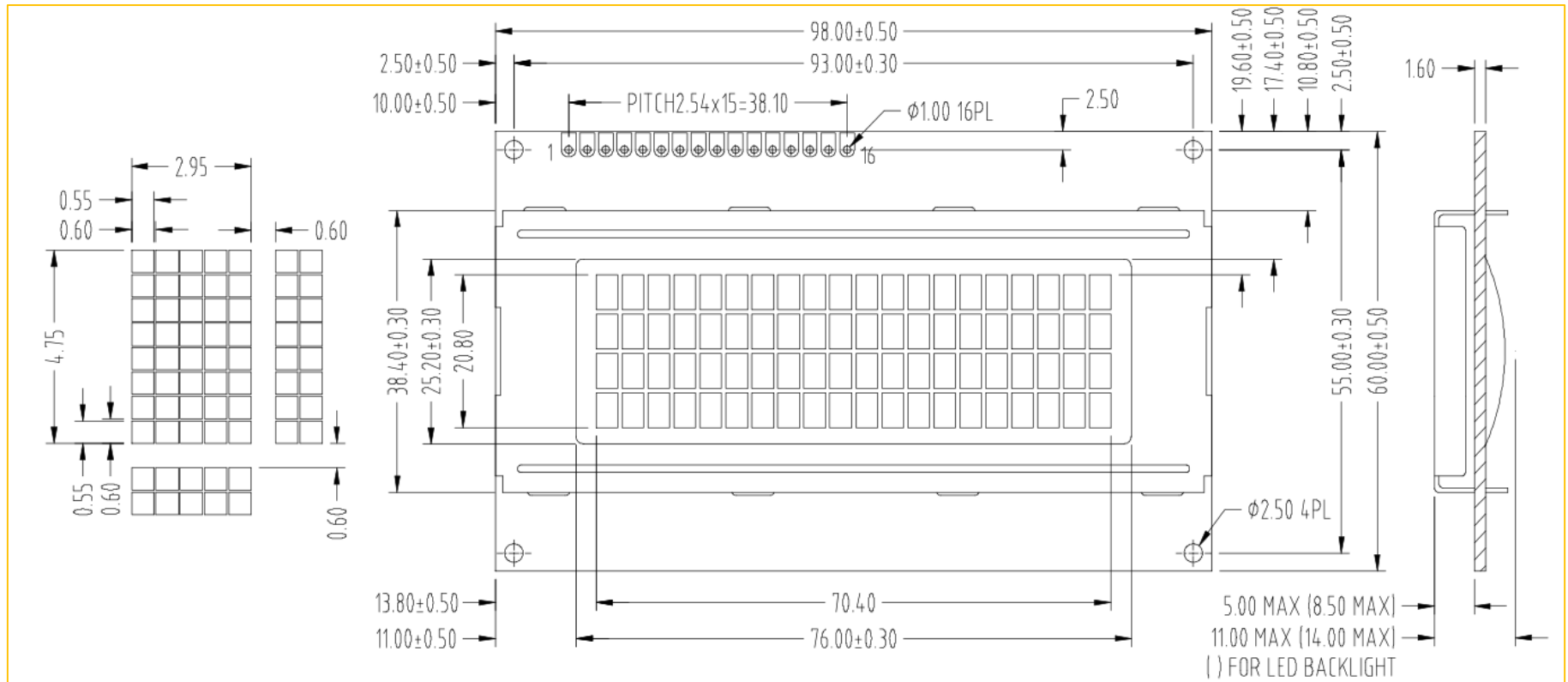
About the Displaytech 204a LCD

Item	Contents	Unit
LCD type	TN / STN / FSTN	---
LCD duty	1/16	---
LCD bias	1/5	---
Viewing direction	6 / 12	o'clock
Module size (W×H×T)	98 × 60 × 11.0 MAX (14.0 MAX W/LED BACKLIGHT)	mm
Viewing area (W×H)	76 × 25.2	mm
Number of characters (characters×lines)	20 × 4	---
Character matrix (W×H)	5 × 8	dots
Character size (W×H)	2.95 × 4.75	mm
Dot size (W×H)	0.55 × 0.55	mm
Dot pitch (W×H)	0.60 × 0.60	mm

LCD: *Viewing Direction*



Displaytech 204a LCD: *External Dimensions*

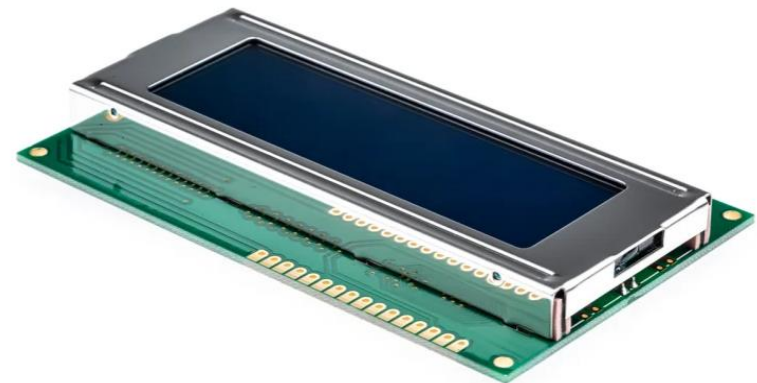
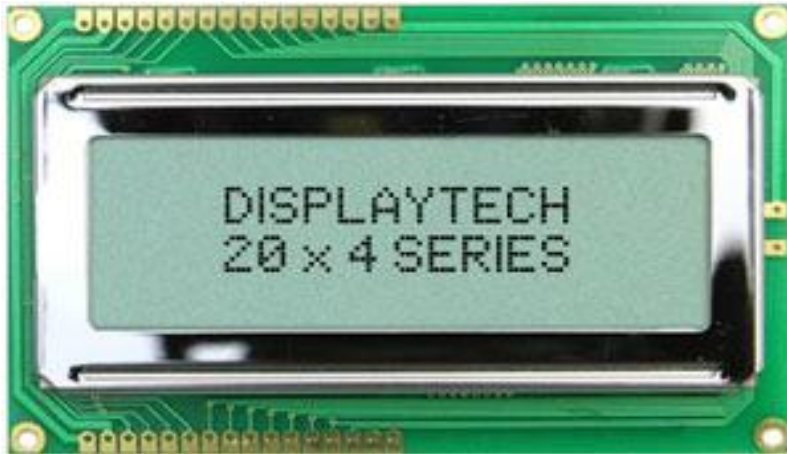


Displaytech 204a LCD: *Actual Picture*

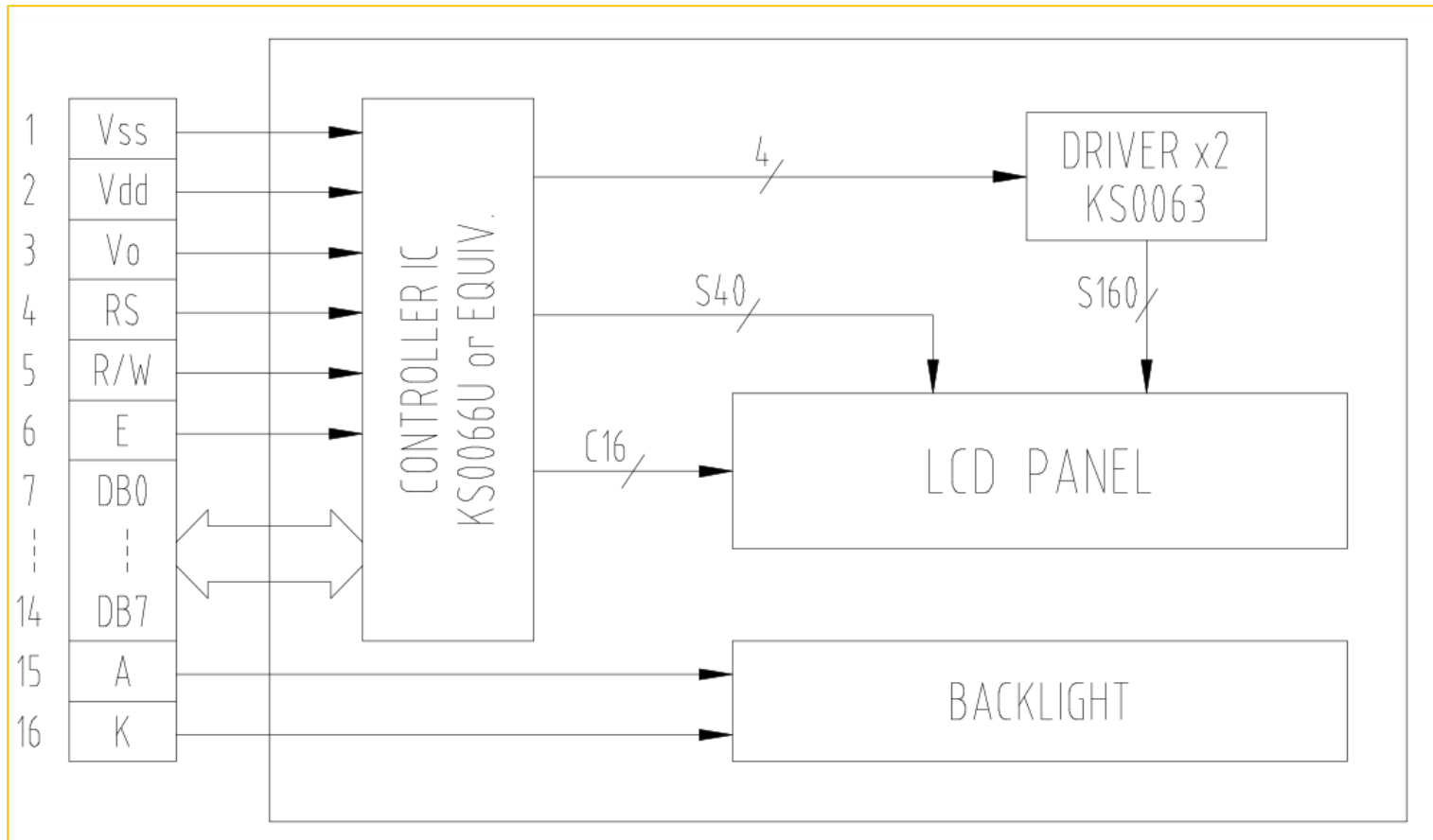


Note: *The above LCD is a negative display type.*

Displaytech 204a LCD: *Actual Picture*



Displaytech 204a LCD: *Block Diagram*



Displaytech 204a LCD: *Interface Signals*

Pin NO.	Symbol	Level	Description
1	VSS	0V	Ground
2	VDD	5.0V	Supply voltage for logic
3	VO	---	Input voltage for LCD
4	RS	H/L	H : Data signal, L : Instruction signal
5	R/W	H/L	H : Read mode, L : Write mode
6	E	H, H → L	Enable signal for KS0076
7	DB0	H/L	Data bit 0
8	DB1	H/L	Data bit 1
9	DB2	H/L	Data bit 2
10	DB3	H/L	Data bit 3
11	DB4	H/L	Data bit 4
12	DB5	H/L	Data bit 5
13	DB6	H/L	Data bit 6
14	DB7	H/L	Data bit 7
15	A	---	Back light anode
16	K	---	Back light cathode

Displaytech 204a LCD: *Registers*

*Instruction Register (IR)**

- stores instruction codes such as “clear display” or “shift cursor” and also stores address information for the display data RAM

*Data Register**

- used for temporarily storing data during data transactions with the MCU

** RS should be logic-0 to access Instruction Register and logic-1 for the Data Register.*

Displaytech 204a LCD: *Registers*

Busy Flag (BF)

- When the busy flag is set at a **logical "1"**, the **LCD unit is executing an internal operation**, and **no instruction will be accepted**

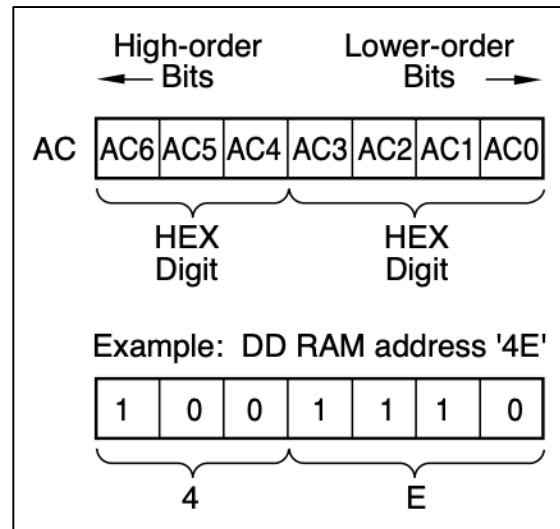
Address Counter (AC)

- The address counter **generates the address for the display data RAM and character generator RAM**

Displaytech 204a LCD: *Registers*

Display Data RAM (DD RAM)

- This **80 x 8 bit RAM** **stores** up to **80 8-bit character codes** as display data



Displaytech 204a LCD: *Registers*

Character Generator ROM (CG ROM)

- This ROM *generates* a **5 × 7 dot-matrix character pattern** for each of **160 different 8-bit character codes**.

Character Generator RAM (CG RAM)

- This RAM *stores* **eight arbitrary 5 x 7 dot-matrix character patterns**, as *programmed by the user*.

HIGH-ORDER 4 BIT LOW- ORDER 4 BIT		0000	0010	0011	0100	0101	0110	0111	1010	1011	1100	1101	1110	1111	
xxxx0000	CG RAM (1)			0	a	P	`	P		-	9	ε	α	p	
xxxx0001	(2)			!	1	A	Q	a	9	7	7	4	ä	q	
xxxx0010	(3)			"	2	B	R	b	r	"	ι	υ	ρ	θ	
xxxx0011	(4)			#	3	C	S	c	s	ι	υ	τ	ε	ω	
xxxx0100	(5)			\$	4	D	T	d	t	,	I	ι	μ	ο	
xxx0101	(6)			%	5	E	U	e	u	*	7	7	1	ç	ù
xxx0110	(7)			&	6	F	V	f	v	7	7	7	ρ	Σ	
xxxx0111	(8)			'	7	G	W	g	w	7	7	7	g	π	
xxxx1000	(1)			(	8	H	X	h	x	ι	υ	7	7	7	
xxxx1001	(2)			)	9	I	Y	i	y	7	7	7	7	7	
xxxx1010	(3)			*	#	J	Z	j	z	ε	ε	ε	j	7	
xxxx1011	(4)			+	;	K	I	k	(	7	7	7	7	7	
xxxx1100	(5)			,	<	L	*	l	ι	υ	7	7	7	7	
xxxx1101	(6)			--	=	M	I	m	>	υ	7	7	7	7	
xxxx1110	(7)			.	>	N	^	n	7	7	7	7	7	7	
xxxx1111	(8)			/	?	O	_	o	ε	υ	7	7	7	7	

Character Codes

Microprocessor Interface

- The **LCD** unit performs **either** dual 4-bit or single 8-bit data transfers, allowing the user to *interface with either a 4-bit or 8-bit microprocessor*

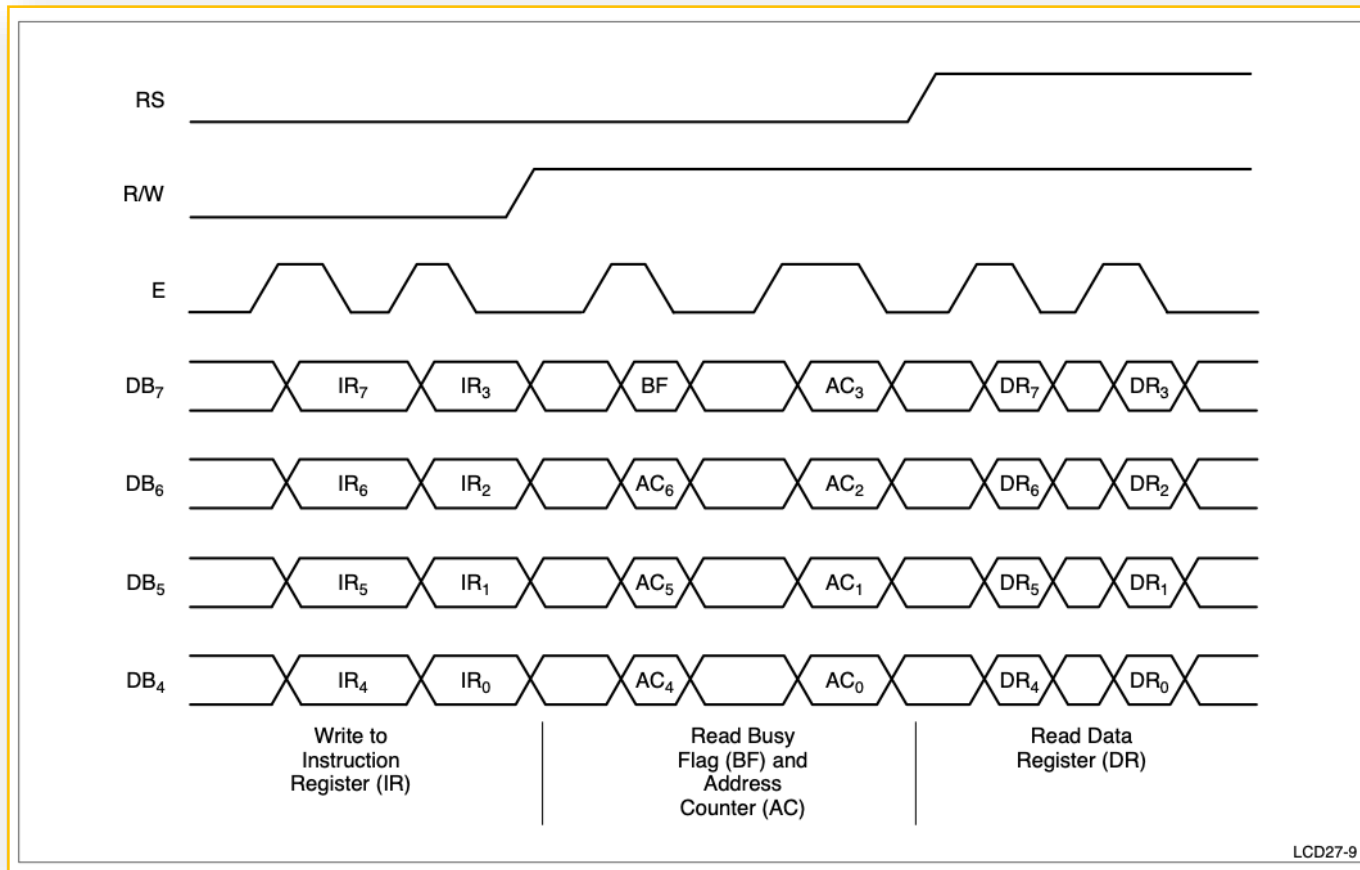
- **4-Bit Microprocessor Interface**
 - Data lines **DB4-DB7** are used for data transfers.
 - Data transactions with the external microprocessor take place in two 4-bit data transfer operations.

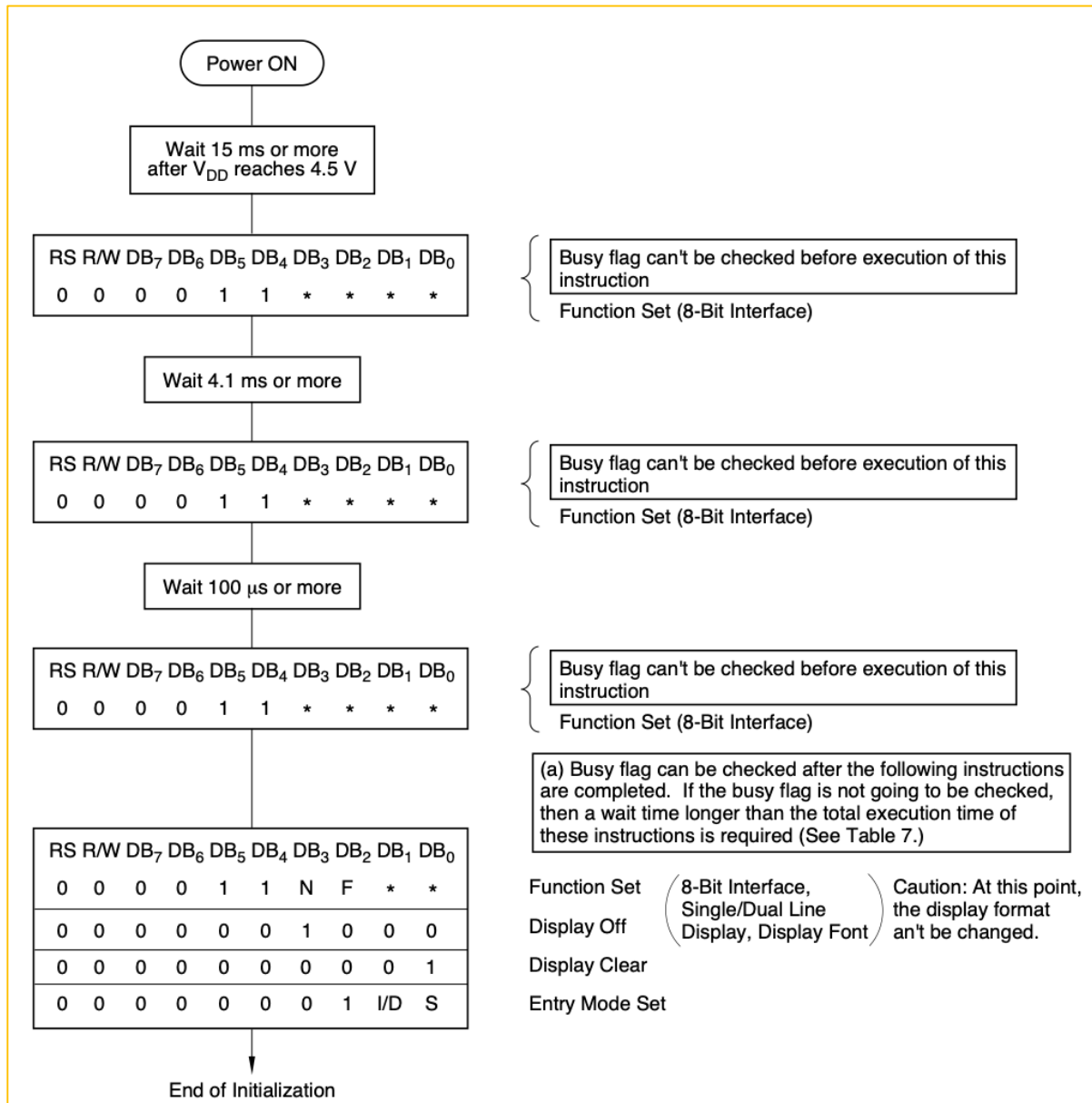
Microprocessor Interface

- **8-bit Microprocessor Interface**
 - Each 8-bit piece of data is transferred in a single operation using the entire data bus DB0-DB7.

Note: *We will be dealing with 8-bit interface only.*

Timing Diagram for the 4-bit Data Transfer





Initialization by Instruction using the 8-bit Interface



Instruction Set

INSTRUCTION	CODE										FUNCTION	EXECUTION TIME (max) (f _{CP} or f _{OSC} = 250 kHz)	
	RS	R/W	DB ₇	DB ₆	DB ₅	DB ₄	DB ₃	DB ₂	DB ₁	DB ₀			
Display Clear	0	0	0	0	0	0	0	0	0	1	Clear enter display area, restore display from shift, and load address counter with DD RAM address 00 _H .	1.64 ms	
Display/Cursor Home	0	0	0	0	0	0	0	0	0	1	*	Restore display from shift and load address counter with DD RAM address 00 _H .	1.64 ms
Entry Mode Set	0	0	0	0	0	0	0	0	1	I/D	S	Specify cursor advance direction and display shift mode. This operation takes place after each data transfer.	40 μs
Display ON/OFF	0	0	0	0	0	0	0	1	D	C	B	Specify activation of display (D), cursor (C), and blinking of character at cursor position (B).	40 μs
Display/Cursor Shift	0	0	0	0	0	0	1	S/C	R/L	*	*	Shift display or move cursor.	40 μs
Function Set	0	0	0	0	0	1	DL	N	0	*	*	Set interface data length (DL) and number of display lines (N).	40 μs
CG RAM Address Set	0	0	0	0	1	ACG					Load the address counter with a CG RAM address. Subsequent data is CG RAM data.	40 μs	
DD RAM Address Set	0	0	0	1	ADD					Load the address counter with a DD RAM address. Subsequent data is DD RAM data.	40 μs		
Busy Flag/Address Counter Read	0	1	BF	AC					Read busy flag (BF) and contents of address counter (AC).	0 μs			
CG RAM/DD RAM Data Write	1	0	Write data					Write data to CG RAM or DD RAM.	40 μs				
CG RAM/DD RAM Data Read	1	1	Read data					Read data from CG RAM or DD RAM.	40 μs				
	I/D = 1: Increment, I/D = 0: Decrement S = 1: Display Shift On S/C = 1: Shift Display, S/C = 0: Move Cursor R/L = 1: Shift Right, R/L = 0: Shift Left DL = 1: 8-Bit, DL = 0: 4-Bit N = 1: Dual Line, N = 0: Single Line BF = 1: Internal Operation, BF = 0: Ready for Instruction										DD RAM: Display Data RAM CG RAM: Character Generator RAM ACG: Character Generator RAM Address ADD: Display Data RAM Address AC: Address Counter		

01H

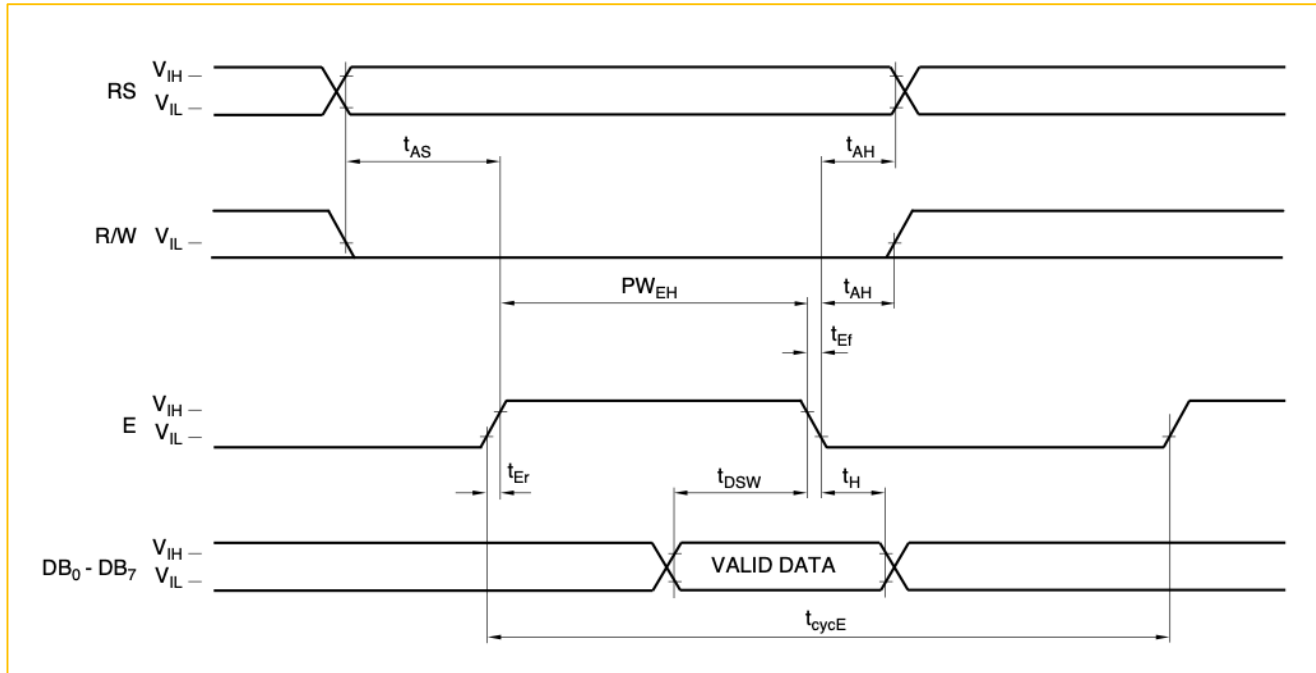
06H

0CH / 08H

38H

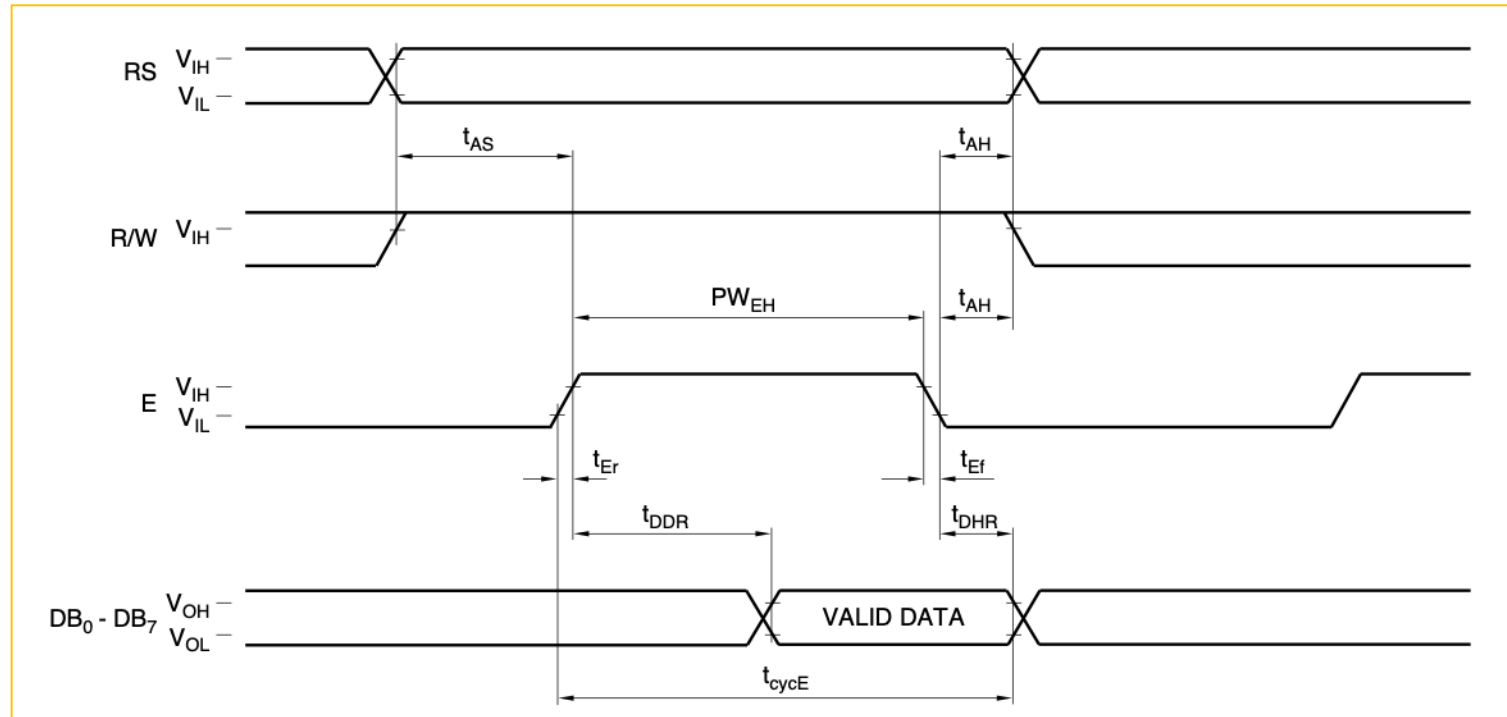


Write Operation Timing Characteristics



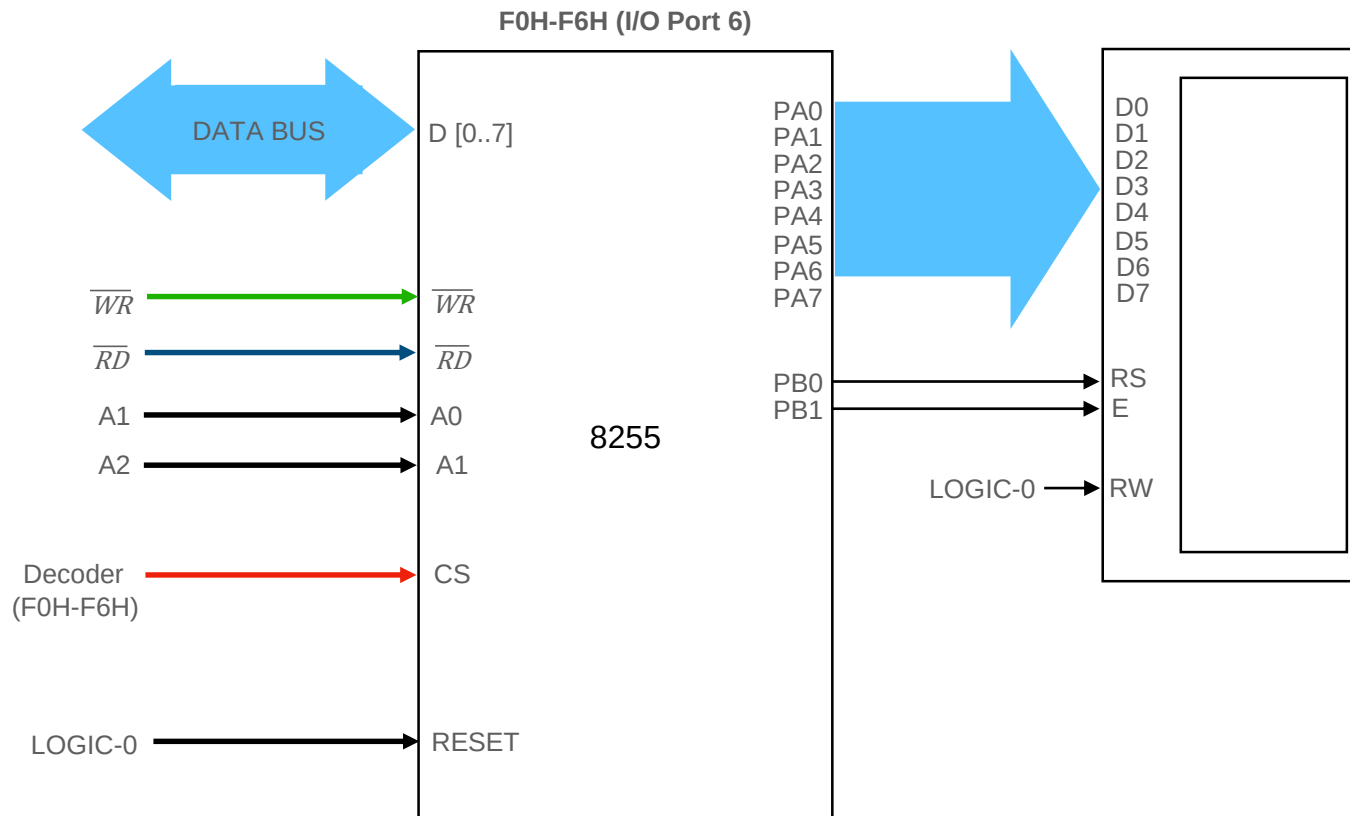
PARAMETER	SYMBOL	VALUE		UNIT
		MIN.	MAX.	
Enable Cycle Time	t_{cycE}	1000	—	ns
Enable Pulse Width "High" Level	PW_{EH}	450	—	ns
Enable Rise/Fall Time	t_{Er}, t_{Erf}	—	25	ns
Setup Time RS, R/W-E	t_{AS}	140	—	ns
Address Hold Time	t_{AH}	10	—	ns
Data Setup Time	t_{DSW}	195	—	ns
Data Hold Time	t_H	10	—	ns

Read Operation Timing Characteristics



PARAMETER	SYMBOL	VALUE		UNIT
		MIN.	MAX.	
Enable Cycle Time	t_{cycE}	1000	—	ns
Enable Pulse Width "High" Level	PW_{EH}	450	—	ns
Enable Rise/Fall Time	t_{Er}, t_{Ef}	—	25	ns
Setup Time RS, R/W-E	t_{AS}	140	—	ns
Address Hold Time	t_{AH}	10	—	ns
Data Delay Time	t_{DDR}	—	320	ns
Data Hold Time	t_{DHR}	20	—	ns

Interfacing via 8255 PPI



Programming

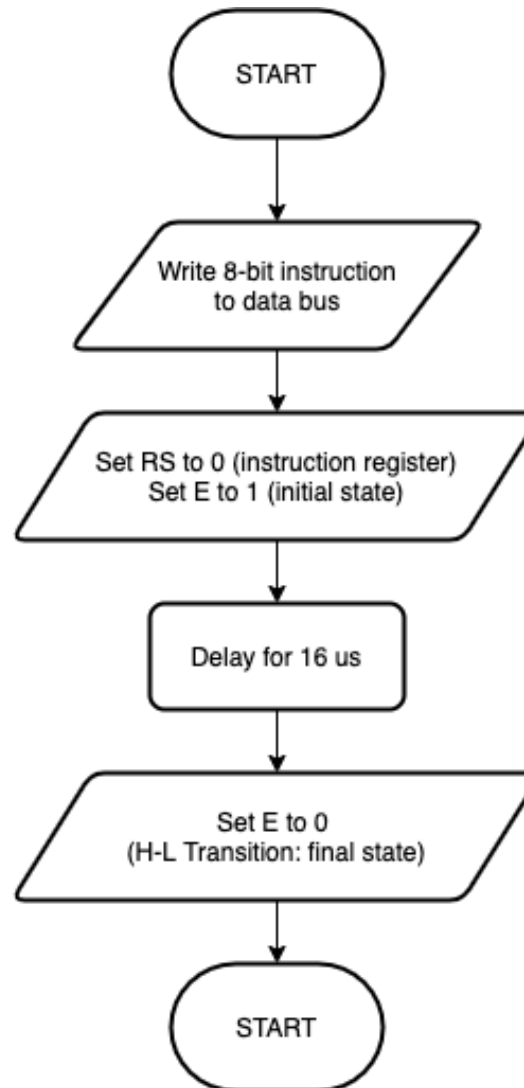
Step 1: Write a function to send Instructions to the LCD

Step 2: Write a function to Initialize the LCD

Step 3: Write a function to send Data to the LCD

Step 4*: Write a function to display a string to the LCD (optional)

Flowchart for Writing 8-bit Instruction to the LCD

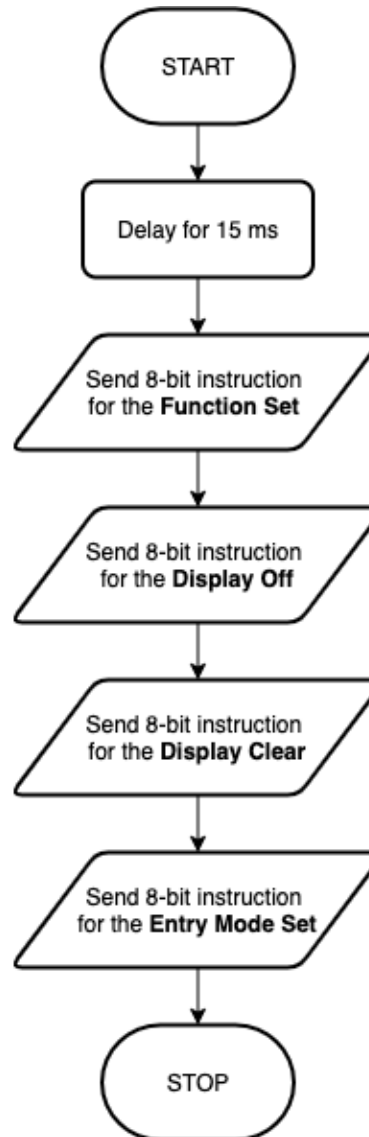


** 16 μ s is the optimal delay for fast instruction response*

INST_CTRL:

```
PUSH AX          ; preserve value of AL
MOV DX, PORTA    ; set port of LCD data bus (PORTA)
OUT DX, AL       ; write data in AL to PORTA
MOV DX, PORTB    ; set port of LCD control lines (PORTB)
MOV AL, 02H      ; E=1, RS=0 (access instruction reg)
OUT DX, AL       ; write data in AL to PORTB
CALL DELAY_1MS   ; delay for 1 ms
MOV DX, PORTB    ; set port of LCD control lines (PORTB)
MOV AL, 00H      ; E=0, RS=0
OUT DX, AL       ; write data in AL to PORTB
POP AX          ; restore value of AL
RET
```

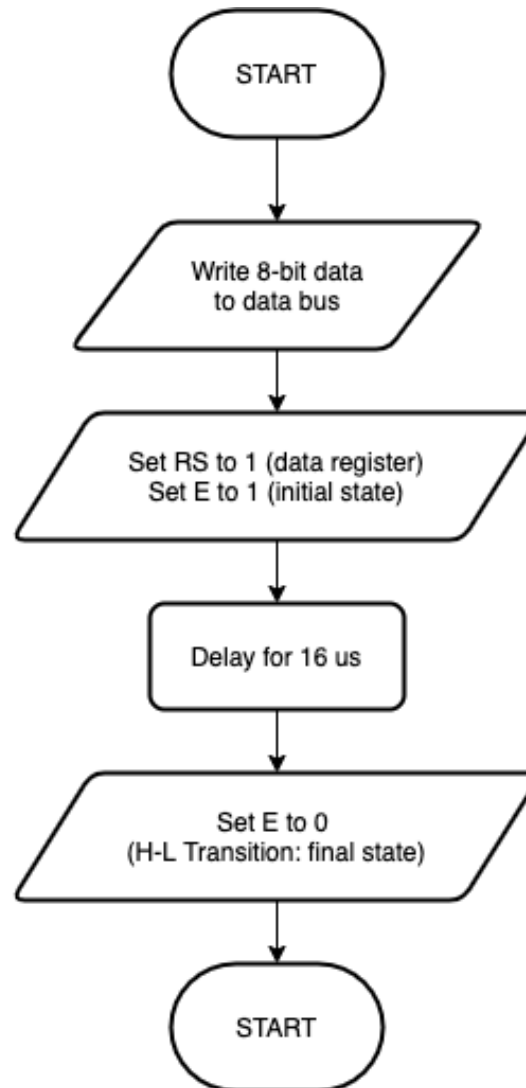
Flowchart for Initializing the LCD



INIT_LCD:

```
MOV AL, 38H      ; 8-bit interface, dual-line display
CALL INST_CTRL  ; write instruction to LCD
MOV AL, 08H      ; display off, cursor off, blink off
CALL INST_CTRL  ; write instruction to LCD
MOV AL, 01H      ; clear display
CALL INST_CTRL  ; write instruction to LCD
MOV AL, 06H      ; increment cursor, display shift off
CALL INST_CTRL  ; write instruction to LCD
MOV AL, 0CH      ; display on, cursor off, blink off
CALL INST_CTRL  ; write instruction to LCD
RET
```

Flowchart in Writing 8-bit Data to the LCD



** 16 us is the optimal delay for fast display refresh*

```
DATA_CTRL:
    PUSH AX          ; preserve value of AL
    MOV DX, PORTA    ; set port of LCD data bus (PORTA)
    OUT DX, AL       ; write data in AL to PORTA
    MOV DX, PORTB    ; set port of LCD control lines (PORTB)
    MOV AL, 03H      ; E=1, RS=1 (access data register)
    OUT DX, AL       ; write data in AL to PORTB
    CALL DELAY_1MS   ; delay for 1 ms
    MOV DX, PORTB    ; set port of LCD control lines (PORTB)
    MOV AL, 01H      ; E=0, RS=1
    OUT DX, AL       ; write data in AL to PORTB
    POP AX           ; restore value of AL
    RET
```

Example 1

Display character '0' at current cursor position:

```
MOV AL, 30H      ; set ASCII code of character  
CALL DATA_CTRL ; display the character
```

Move the cursor at position at third column, first line:

```
MOV AL, 0C2H     ; set address of third column, first line*  
CALL INST_CTRL  ; send instruction to move cursor to position
```

* see character display address

Example 2

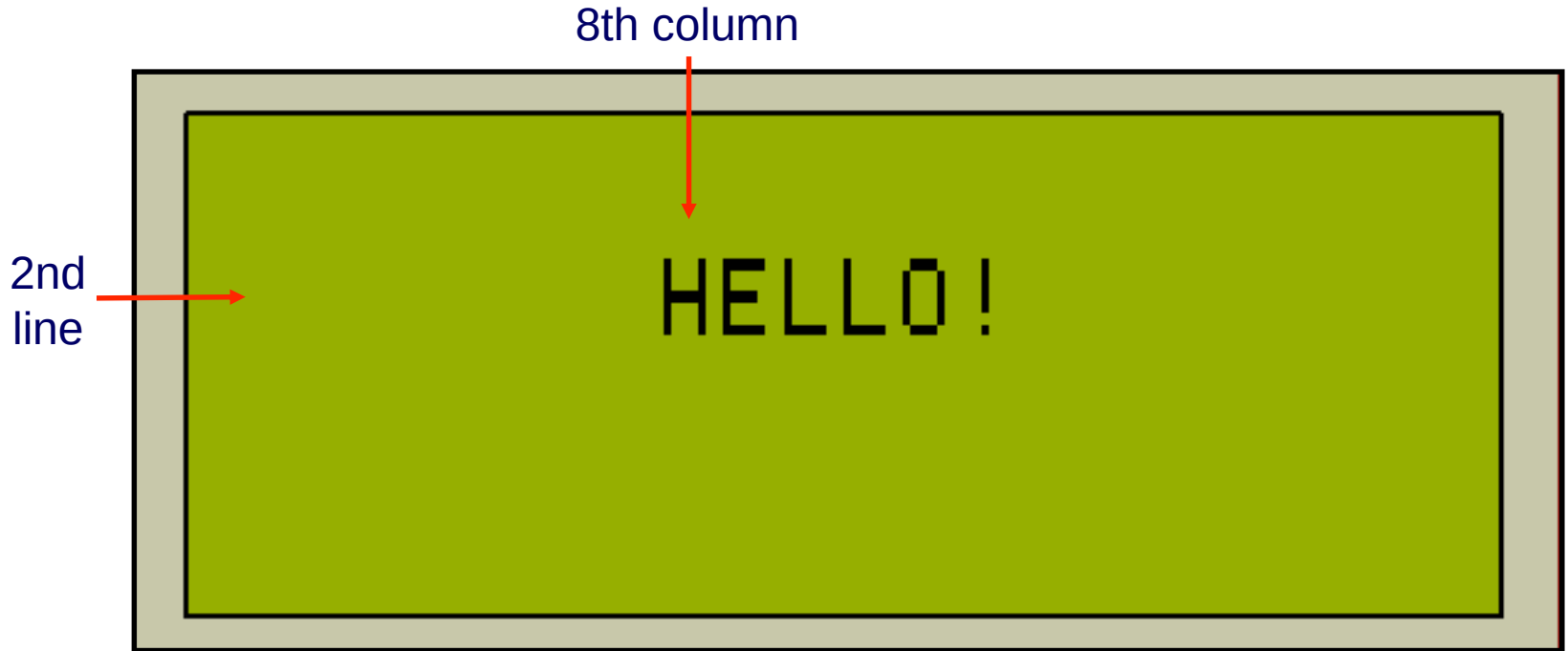
Display string “HELLO!” at the middle of second line:

```

MOV AL, 0C7H      ; move cursor to 8th column of 2nd line
CALL INST_CTRL    ; send instruction to LCD
MOV AL, 'H'        ; set ASCII code of character 'H'
CALL DATA_CTRL   ; display the character
MOV AL, 'E'        ; set ASCII code of character 'E'
CALL DATA_CTRL   ; display the character
MOV AL, 'L'        ; set ASCII code of character 'L'
CALL DATA_CTRL   ; display the character
MOV AL, 'L'        ; set ASCII code of character 'L'
CALL DATA_CTRL   ; display the character
MOV AL, 'O'        ; set ASCII code of character 'O'
CALL DATA_CTRL   ; display the character
MOV AL, '!'        ; set ASCII code of character '!'
CALL DATA_CTRL   ; display the character

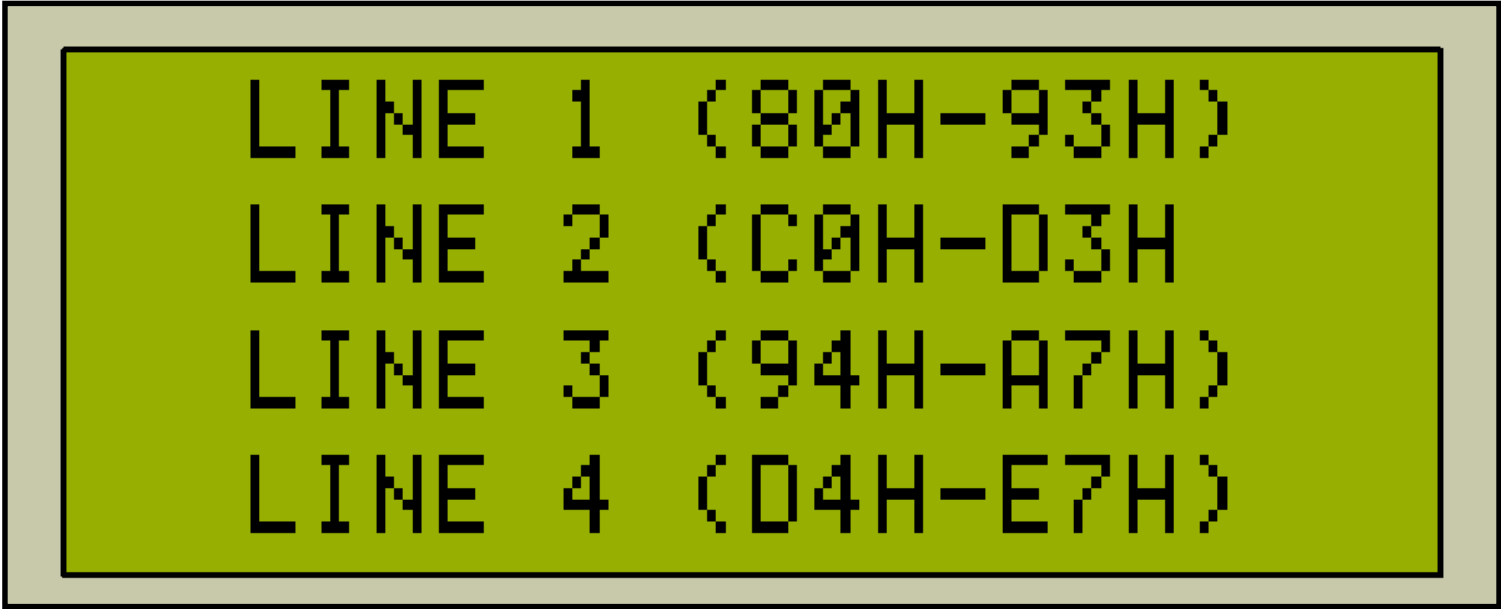
```

Example 2



String displayed beginning at 8th column of the second line.

Character Display Addresses



LINE	1	(80H-93H)
LINE	2	(C0H-D3H)
LINE	3	(94H-A7H)
LINE	4	(D4H-E7H)

Line 1 address continue to Line 3 while Line 2 continues at Line 4.

Exercise

- The LCD will be interfaced to the 8086 μ P via the 8255 PPI at address F0H-F6H (see slide 22).
- **LCD configuration:** 8-bit, cursor off, blink off, increment cursor, display shift off
- LCD is initialized first, then “HELLO WORLD!” will be displayed at the middle of the 3rd line.

```

DATA SEGMENT
    PORTA EQU 0F0H          ; address to access PORTA
    PORTB EQU 0F2H          ; address to access PORTB
    PORTC EQU 0F4H          ; address to access PORTC
    PORT_CON EQU 0F6H       ; address to access Command Register
DATA ENDS

CODE SEGMENT PUBLIC 'CODE'
    ASSUME CS:CODE
START:
    ; Program 8255 (PORTA & PORTB as output, PORTC as input)
    MOV DX, PORT_CON        ; set port to Command Register
    MOV AL, 89H             ; set command byte

    OUT DX, AL              ; send data in AL to Command Register

    CALL INIT_LCD           ; initialize LCD
    MOV AL, 98H             ; move cursor to 3rd line, 5th column

    CALL INST_CTRL          ; send instruction to LCD
    ; display string "HELLO WORLD!" at current cursor position
    MOV AL, 'H'             ; set ASCII code of character 'H'
    CALL DATA_CTRL ; display the character
    MOV AL, 'E'             ; set ASCII code of character 'E'
    CALL DATA_CTRL ; display the character
    MOV AL, 'L'             ; set ASCII code of character 'L'
    CALL DATA_CTRL ; display the character
    MOV AL, 'L'             ; set ASCII code of character 'L'
    CALL DATA_CTRL ; display the character
    MOV AL, 'O'             ; set ASCII code of character 'O'
    CALL DATA_CTRL ; display the character
    MOV AL, ' '             ; set ASCII code of character ' '
    CALL DATA_CTRL ; display the character
    MOV AL, 'W'             ; set ASCII code of character 'W'
    CALL DATA_CTRL ; display the character
    MOV AL, 'O'             ; set ASCII code of character 'O'
    CALL DATA_CTRL ; display the character
    MOV AL, 'R'             ; set ASCII code of character 'R'
    CALL DATA_CTRL ; display the character

```

```

    MOV AL, 'L'      ; set ASCII code of character 'L'
    CALL DATA_CTRL ; display the character
    MOV AL, 'D'      ; set ASCII code of character 'D'
    CALL DATA_CTRL ; display the character
    MOV AL, '!'      ; set ASCII code of character '!'

    CALL DATA_CTRL ; display the character
ENDLESS:

    JMP ENDLESS
; this function will send an 8-bit instruction to the LCD
INST_CTRL:
    PUSH AX          ; preserve value of AL
    MOV DX, PORTA    ; set port of LCD data bus (PORTA)
    OUT DX, AL        ; write data in AL to PORTA
    MOV DX, PORTB    ; set port of LCD control lines (PORTB)
    MOV AL, 02H      ; E=1, RS=0 (access instruction reg)
    OUT DX, AL        ; write data in AL to PORTB
    CALL DELAY_1MS   ; delay for 1 ms
    MOV DX, PORTB    ; set port of LCD control lines (PORTB)
    MOV AL, 00H      ; E=0, RS=0
    OUT DX, AL        ; write data in AL to PORTB
    POP AX           ; restore value of AL

    RET
; this function will send an 8-bit data (character) to the LCD
DATA_CTRL:
    PUSH AX          ; preserve value of AL
    MOV DX, PORTA    ; set port of LCD data bus (PORTA)
    OUT DX, AL        ; write data in AL to PORTA
    MOV DX, PORTB    ; set port of LCD control lines (PORTB)
    MOV AL, 03H      ; E=1, RS=1 (access data reg)
    OUT DX, AL        ; write data in AL to PORTB
    CALL DELAY_1MS   ; delay for 1 ms
    MOV DX, PORTB    ; set port of LCD control lines (PORTB)
    MOV AL, 01H      ; E=0, RS=1
    OUT DX, AL        ; write data in AL to PORTB
    POP AX           ; restore value of AL

    RET

```

```

        ; this function will initialize the LCD
INIT_LCD:
    MOV AL, 38H      ; 8-bit interface, dual-line display
    CALL INST_CTRL ; write instruction to LCD
    MOV AL, 08H      ; display off, cursor off, blink off
    CALL INST_CTRL ; write instruction to LCD
    MOV AL, 01H      ; clear display
    CALL INST_CTRL ; write instruction to LCD
    MOV AL, 06H      ; increment cursor, display shift off
    CALL INST_CTRL ; write instruction to LCD
    MOV AL, 0CH      ; display on, cursor off, blink off
    CALL INST_CTRL ; write instruction to LCD

    RET
DELAY_1MS:
    MOV BX, 02CAH
L1:
    NOP
    JNZ
    L1

    RET
CODE ENDS
END START

```



University of San Carlos | Department of
COMPUTER ENGINEERING

CpE 3104 – Microprocessors

End of Lecture

UNIT 7: INTERFACING PARALLEL I/O DEVICES

© 2025